

# Documentation Complete du Backend API (FastAPI)

---

Projet : **MSPR - ObRail Europe**

Version : backend actuel ([/backend/app](#))

Technologies : FastAPI, SQLAlchemy, PostgreSQL, Pydantic

---

## 1. Objectif du backend

Le backend expose une API REST permettant :

- la consultation des donnees ferroviaires (trajets, gares, lignes),
- la production de statistiques de pilotage (KPI),
- l'alimentation du dashboard Streamlit.

Le service est concu en architecture modulaire : [routes](#) -> [services](#) -> [models](#) -> [database](#).

---

## 2. Architecture technique

### 2.1 Structure des dossiers backend

```
backend/  
├── app/  
│   ├── main.py  
│   ├── database.py  
│   ├── routes/  
│   ├── services/  
│   ├── models/  
│   └── schemas/  
└── requirements.txt
```

### 2.2 Role de chaque couche

- [main.py](#) : instancie FastAPI et enregistre les routers.
  - [database.py](#) : cree l'engine SQLAlchemy, la session DB et [Base](#).
  - [models/](#) : mapping ORM des tables PostgreSQL.
  - [schemas/](#) : schema de reponse Pydantic.
  - [services/](#) : logique metier / acces base.
  - [routes/](#) : endpoints HTTP.
- 

## 3. Prerequis et lancement

### 3.1 Dependances Python

Fichier [backend/requirements.txt](#) :

- fastapi
- uvicorn
- sqlalchemy
- psycopg2-binary
- pydantic
- python-dotenv
- alembic
- pytest
- httpx

### 3.2 Variable d'environnement requise

Fichier `backend/.env` :

```
DATABASE_URL=postgresql://postgres:<mot_de_passe>@localhost:5432/obrail
```

### 3.3 Lancement local

```
cd /home/johan/MSPR/backend
source ../venv/bin/activate
uvicorn app.main:app --reload --port 8000
```

### 3.4 URLs utiles

- API base : `http://127.0.0.1:8000`
- Swagger UI : `http://127.0.0.1:8000/docs`
- ReDoc : `http://127.0.0.1:8000/redoc`
- OpenAPI JSON : `http://127.0.0.1:8000/openapi.json`

---

## 4. Configuration FastAPI

`app/main.py` enregistre les routers suivants :

- `health_routes`
- `trajet_routes`
- `gare_routes`
- `ligne_routes`
- `stats_routes`

Tous les endpoints sont donc exposes dans un seul service FastAPI.

---

## 5. Connexion base de donnees

Fichier : `app/database.py`

- `create_engine(DATABASE_URL)` : connexion PostgreSQL
- `SessionLocal` : sessions SQLAlchemy
- `Base = declarative_base()` : base ORM
- `get_db()` : dependency FastAPI pour injecter une session DB proprement

Comportement :

- ouverture session au debut de la requete
- fermeture automatique en `finally`

---

## 6. Modeles de donnees principaux (ORM)

Modeles utilises activement par les routes :

- `Trajet` (`trajet`)
- `Gare` (`gare`)
- `Ligne` (`ligne`)
- `Operateur` (`operateur`)
- `Emission` (`emission`)
- `Itineraire` (`itineraire`)

Relations majeures :

- `Trajet.id_ligne` -> `Ligne.id_ligne`
- `Itineraire.trajet_id` -> `Trajet.trajet_id`
- `Itineraire.code_uic` -> `Gare.code_uic`

---

## 7. Schemas de reponse (Pydantic)

### 7.1 `TrajetResponse`

```
{
  "trajet_id": "string",
  "id_ligne": 0,
  "gare_depart": "string",
  "gare_arrivee": "string",
  "heure_depart": "string",
  "heure_arrivee": "string"
}
```

### 7.2 `GareResponse`

```
{
  "code_uic": "string",
  "nom_gare": "string",
  "longitude": 0.0,
  "latitude": 0.0,
```

```
"iso_pays": "string"
}
```

### 7.3 LigneResponse

```
{
  "id_ligne": 0,
  "nom_ligne": "string",
  "distance": 0.0,
  "type_service": "JOUR|NUIT|null"
}
```

---

## 8. Documentation complete des endpoints

### 8.1 Sante

#### GET /health

Controle de disponibilite du service.

Reponse :

```
{"status": "ok"}
```

---

### 8.2 Trajets

#### GET /trajets/

Retourne la liste complete des trajets.

- Response model : `List[TrajetResponse]`

Exemple :

```
curl http://127.0.0.1:8000/trajets/
```

#### GET /trajets/{trajet\_id}

Retourne un trajet par identifiant.

- Response model : `TrajetResponse`
- Erreur : `404 Trajet not found`

Exemple :

```
curl "http://127.0.0.1:8000/trajets/SNCF%20IC%20Nuit%203755%20(Sun)"
```

---

## 8.3 Gares

GET /gares/

Retourne la liste de toutes les gares.

- Response model : `List[GareResponse]`

Exemple :

```
curl http://127.0.0.1:8000/gares/
```

---

## 8.4 Lignes

GET /lignes/

Retourne la liste de toutes les lignes.

- Response model : `List[LigneResponse]`

Exemple :

```
curl http://127.0.0.1:8000/lignes/
```

---

## 8.5 Statistiques (dashboard)

GET /stats/trajets/count

Retour :

```
{"total_trajets": 15482}
```

GET /stats/lignes/count

Retour :

```
{"total_lignes": 343}
```

### GET /stats/gares/count

Retour :

```
{"total_gares": 23770}
```

### GET /stats/trajets/type

Repartition des trajets par type de service.

Retour :

```
{"JOUR": 6845, "NUIT": 4419}
```

### GET /stats/emissions

Moyenne des empreintes CO2.

Retour :

```
{"train": 125.0, "avion": 932.0}
```

### GET /stats/operateurs

Volumes par operateur.

Retour (extrait) :

```
[  
  {"operateur": "SNCF", "trajets": 1800},  
  {"operateur": "DB", "trajets": 1200}  
]
```

### GET /stats/trajets/map

Segments geographiques pour la carte.

Retour (extrait) :

```
[
  {
    "lat_depart": 48.8534,
    "lon_depart": 2.3488,
    "lat_arrivee": 50.6292,
    "lon_arrivee": 3.0573
  }
]
```

Note implementation :

- selection d'un echantillon de trajets (limite),
- reconstruction des troncons via `itineraire` + `gare`,
- deduplication des segments identiques.

---

## 9. Gestion des erreurs et codes HTTP

Codes observes :

- `200 OK` : succes
- `307 Temporary Redirect` : appel sans slash terminal (FastAPI redirige vers `/.../`)
- `404 Not Found` : trajet inexistant (`/trajets/{id}`)
- `500 Internal Server Error` : erreur serveur/DB (a investiguer via logs)

Bonnes pratiques :

- toujours utiliser les URLs avec slash final pour les listes (`/gares/`, `/trajets/`, `/lignes/`)
- monitorer les logs Uvicorn en environnement de dev

---

## 10. Contrat backend <-> dashboard

Le dashboard depend explicitement de :

- format JSON des endpoints `/stats/*`,
- schema `GareResponse` pour la carte,
- schema `TrajetResponse` et `LigneResponse` pour les pages analytiques.

Tout changement de nom de champ dans l'API doit etre repercute dans :

- `dashboard/services/api_service.py`
- `dashboard/app.py`
- `dashboard/components/*.py`

---

## 11. Verification rapide (smoke tests)

Executer ces commandes apres demarrage backend :

```
curl http://127.0.0.1:8000/health
curl http://127.0.0.1:8000/stats/trajets/count
curl http://127.0.0.1:8000/stats/trajets/type
curl http://127.0.0.1:8000/stats/emissions
curl http://127.0.0.1:8000/stats/trajets/map
```

Resultat attendu :

- pas d'erreur 500,
- JSON valide sur chaque endpoint.

---

## 12. Evolutions recommandees (prochaine iteration)

1. Ajouter une couche **CRUD** plus propre avec injection DB uniforme (`Depends(get_db)`) dans tous les services.
2. Ajouter des schemas de reponse explicites pour les endpoints `/stats/*`.
3. Ajouter pagination/limit sur `/trajets/` et `/gares/`.
4. Ajouter filtres query params (`type_service`, `pays`, `opérateur`).
5. Ajouter tests API automatisés (pytest + httpx) endpoint par endpoint.
6. Ajouter gestion centralisée des erreurs (handlers FastAPI).

---

## 13. Resume executif

Le backend FastAPI est operationnel, structure et deja exploitable pour le dashboard.

Il expose un noyau solide d'endpoints de consultation et de statistiques, base sur SQLAlchemy/PostgreSQL.

La documentation Swagger est disponible nativement et cette documentation detaille les contrats API, formats de reponse et bonnes pratiques d'exploitation.